Chyba 1 – Content Security Policy (CSP) chyběla, ztráta 25 bodů Web neměl nastaveno žádné pravidlo o tom co smí prohlížeč načítat. To znamenalo, že útočník mohl potenciálně vložit cizí skripty a provést XSS útok.

Řešení: Do `app.py` jsem přidal hlavičku která explicitně povoluje pouze vlastní zdroje a Google Fonts. JavaScript je zakázán úplně:

python
```
response.headers['Content-Security-Policy'] = (
  "default-src 'self'; "
  "style-src 'self' 'unsafe-inline' https://fonts.googleapis.com; "
  "font-src 'self' https://fonts.gstatic.com; "
  "script-src 'none';"
)
```

Chyba 2 – Přesměrování HTTP → HTTPS chybělo, ztráta 20 bodů Web byl dostupný přes nezabezpečené HTTP, takže útočník mohl odposlouchávat přenášená data.

Řešení: Dvě věci naráz. V `app.py` jsem přidal funkci která detekuje HTTP požadavek a přesměruje na HTTPS:

python
```
@app.before_request
def redirect_https():
  if request.headers.get('X-Forwarded-Proto') == 'http':
    return redirect(request.url.replace('http://', 'https://'), code=301)
```

A zároveň jsem na PythonAnywhere v záložce **Web → Security** zapnul přepínač **Force HTTPS**.

Chyba 3 – Referrer-Policy chyběla Při kliknutí na odkaz z webu se odesílala celá URL adresa včetně případných citlivých parametrů na cílový server.

Řešení: Přidal jsem hlavičku která omezuje odesílání referrer informací pouze na původ (doménu) bez celé URL:

python
```
response.headers['Referrer-Policy'] = 'strict-origin-when-cross-origin'
```

Chyba 4 – Strict Transport Security (HSTS) chyběla, ztráta 20 bodů Prohlížeč nebyl donucen vždy používat HTTPS. I když HTTPS existovalo, první připojení mohlo proběhnout přes HTTP.

Řešení: Přidal jsem HSTS hlavičku která říká prohlížeči aby si zapamatoval na celý rok, že tento web smí načítat pouze přes HTTPS:

python
```
response.headers['Strict-Transport-Security'] = 'max-age=31536000; includeSubDomains'
```

Chyba 5 – Subresource Integrity Tato chyba se mě netýkala, protože web neobsahuje žádné externí skripty. Observatory to samo uznalo jako `None` bez ztráty bodů.

Chyba 6 – X-Content-Type-Options chyběla, ztráta 5 bodů Prohlížeč mohl sám hádat typ souboru místo toho aby věřil serveru. Například mohl spustit textový soubor jako JavaScript.

Řešení: Přidal jsem hlavičku která prohlížeči zakáže hádat typ souboru:

python
```
response.headers['X-Content-Type-Options'] = 'nosniff'
```

Chyba 7 – X-Frame-Options chyběla, ztráta 20 bodů Web šlo vložit do iframe na cizí stránce a provést clickjacking útok – uživatel si myslí že kliká na jednu věc ale ve skutečnosti kliká na něco jiného.

Řešení: Přidal jsem hlavičku která zakazuje vkládání webu do jakéhokoliv iframe:

python
```
response.headers['X-Frame-Options'] = 'DENY'
```

Všechny hlavičky jsou přidávány centrálně jednou funkcí v `app.py` pomocí `@app.after_request` dekorátoru, takže se automaticky aplikují na každou stránku webu bez toho abych musel měnit jednotlivé šablony.

8 Subresource Integrity – i když se tě netýká, pro pochopení:

Tvůj web načítá Google Fonts takto:

html
```
<link href="https://fonts.googleapis.com/css2?family=Syne..." rel="stylesheet">
```

Toto je externí zdroj – soubor který nepochází z tvého serveru ale z Googlu.

Bez SRI: Prohlížeč stáhne soubor z Googlu a důvěřuje mu slepě – i kdyby byl změněn.

Se SRI: Prohlížeč si spočítá otisk staženého souboru a porovná ho s hodnotou kterou jsi zadal v HTML. Pokud se neshodují – soubor odmítne načíst.

Proč se tě to netýká:

Observatory kontroluje pouze **JavaScript soubory** – ty jsou nebezpečné protože se spouštějí jako kód. Tvůj web žádný externí JS nenačítá, takže není co kontrolovat.

Google Fonts je pouze CSS (styly vzhledu) – to Observatory ignoruje, proto dostals hodnocení `None` místo `Failed`.